

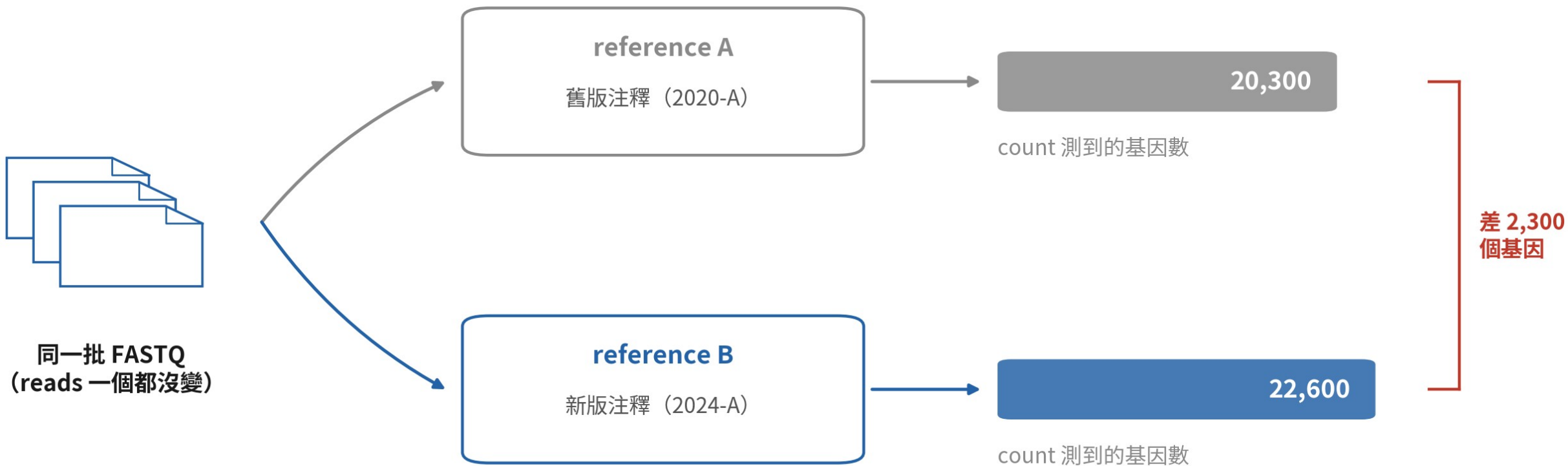
B2 B 系列

參考基因組與 FASTQ： mkref 與 mkfastq

看懂 FASTQ 三個檔案各存什麼、reference 從哪來，需要自建時知道怎麼建。

約 35 分鐘 | 前置：B1 | 資料：PBMC 1k FASTQ (10x 官方)

同一批 FASTQ ，兩個 reference



示意數字

reads 沒變、程式沒變——差的全在 reference。差在哪？

reads 一個都沒變，測到的基因數卻差了兩千多個——差在哪？

本集的起點

**count 還沒開始跑，
你的結果就已經被決定了一半。**

FASTQ 對不對、reference 對不對——這兩件事出錯，下游做得再漂亮都是白工。

這一集你會學到

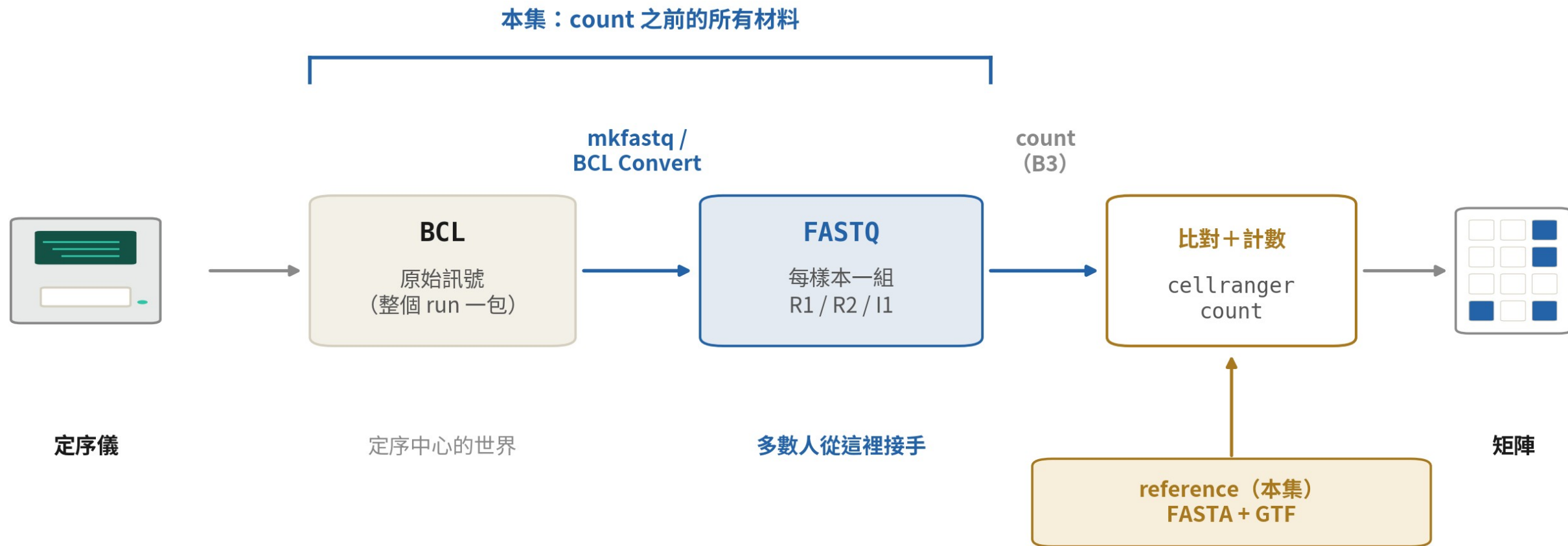
- 1 說出 R1 / R2 / I1 各存什麼、長度為什麼是那樣
- 2 判斷什麼時候用 10x 預建 reference、什麼時候必須 mkref 自建
- 3 讀懂 GTF 與 filter 對「哪些基因會被計數」的影響

01

從定序儀到矩陣

先看懂整條資料流，再決定哪幾步輪得到你

count 之前的資料流



BCL → FASTQ → (比對+計數) → 矩陣; reference 從旁邊餵進 count

reference = 序列 + 注釋，兩件事

genome.fa (FASTA) ——地圖本身

>chr1

AGCTTACCGTTAGCCTAAGGCTTACGATCC...

每條染色體的完整序列，31 億個字母

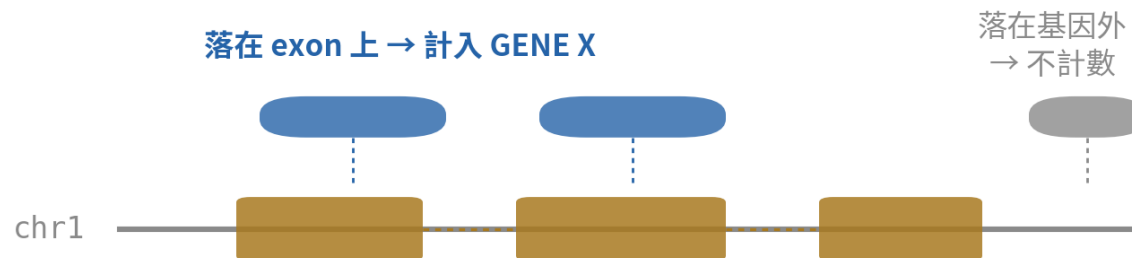
genes.gtf (GTF) ——地圖上的地標

chr1 gene 11869 14409 DDX11L1

chr1 exon 11869 12227 ...

哪一段座標屬於哪個基因、哪裡是 exon

兩個檔案合作，一條 read 才數得進基因



GTF：這三段 exon 合起來叫 GENE X

FASTA 決定「read 能比對到哪裡」
GTF 決定「那個位置算不算、算給誰」

版本必須成對：同一份基因組、同一版座標

- FASTA：基因組序列—— reads 能「比對到哪」
- GTF：基因注釋——那個位置「算不算、算給誰」
- 兩者版本必須成對，座標系統才一致
- cold open 的差異，就藏在 GTF 那一半

先放下焦慮

這條流程的大半， 多數人一輩子不用自己跑。

定序中心通常交給你現成的 FASTQ ；人與小鼠有預建 reference 。但每個檔案你都要看得懂——出事的時候，看不懂的人只能重測。

02

裝好 Cell Ranger

Linux 限定、記憶體 64 GB 起跳、下載要註冊

系統需求：先看你的機器夠不夠格



下載也有門檻：要先在 10x 官網註冊，網站才給你（限時的）下載連結

- Linux x86-64 限定——筆電跑不動就借伺服器
- 記憶體 64 GB 起跳，是人類資料的硬底線
- 下載連結要先在 10x 官網註冊才拿得到

下載、解壓、加進 PATH

```
# 10x 官網註冊後，網站會給你一段帶簽名的 curl 指令，長這樣：  
curl -o cellranger-9.0.1.tar.gz "https://cf.10xgenomics.com /...."  
  
tar -xzf cellranger-9.0.1.tar.gz  
export PATH=$PWD /cellranger-9.0.1:$PATH # 寫進 ~ /.bashrc 才常駐  
  
cellranger --version
```

```
cellranger cellranger-9.0.1
```

下載連結有時效，過期回官網重新產生；版本號以你下載當下為準

驗收安裝: testrun

```
cellranger testrun --id=check_install  
# 用內建的迷你資料集跑一次完整流程, 約 5-10 分鐘
```

```
Martian Runtime - v4.x  
Running preflight checks (please wait)...  
...  
Pipestance completed successfully!
```

你要看的是最後一行: Pipestance completed successfully

03

FASTQ 解剖

三個檔案、三個問題：誰、什麼、哪個樣本

一筆 read ，固定四行

```
@A00228:279:HFVFVDMXX:1:1101:8828:1000 1:N:0:ACGTACGT
```

```
CTACATTCAGCCACCAGGTGTGAGTTACGCAT
```

```
+
```

```
FFFFFFFF:FFFFFFFFFFFFFF,FFFFFFFF
```

—— 第 1 行：read ID——機器、run、flowcell、座標

—— 第 2 行：序列本體 (A/C/G/T/N)

—— 第 3 行：分隔線，固定一個加號

—— 第 4 行：品質分數——每個字元對應上面一個鹼基

一筆 read = 固定四行；一個 FASTQ 檔就是幾千萬筆這樣的紀錄

FASTQ 只是把幾千萬筆「四行紀錄」壓縮起來的文字檔

R1、R2、I1 各存什麼

三種檔案，三個問題：誰（R1）、什麼（R2）、哪個樣本（I1/I2）

R1

cell barcode 16 bp

哪一顆細胞

UMI 12 bp

哪一個分子

= 28 bp，一個鹼基都不多

R2

cDNA 片段（約 90 bp）——真正拿去比對基因組的讀段

哪一個基因

I1/I2

10 bp

哪一個樣本

sample index——哪一個樣本（mkfastq 分揀時用，count 不用）

R1 的長度就是 chemistry 的身分證：28 bp = v3/v4 (16+12)、26 bp = v2 (16+10)

這是 10x 的定義，跟 bulk RNA-seq 的 R1/R2 完全不同——本集最大的坑

打開 R1 看一眼

```
zcat pbm_c_1k_v3_S1_L001_R1_001.fastq.gz | head -4
```

```
@ A00228:279:HFV FVDMXX:1:1101:8828:1000 1:N:0:ACGTACGT  
CTACATTCAGCCACCAGGTGTGAGTTAC  
+  
FFFFFFFF:FFFFFFFFFFFFFFF,FFFFF
```

第二行剛好 28 個字母：前 16 是 barcode、後 12 是 UMI——它不是拿去比對的序列

檢查讀長：一行 awk

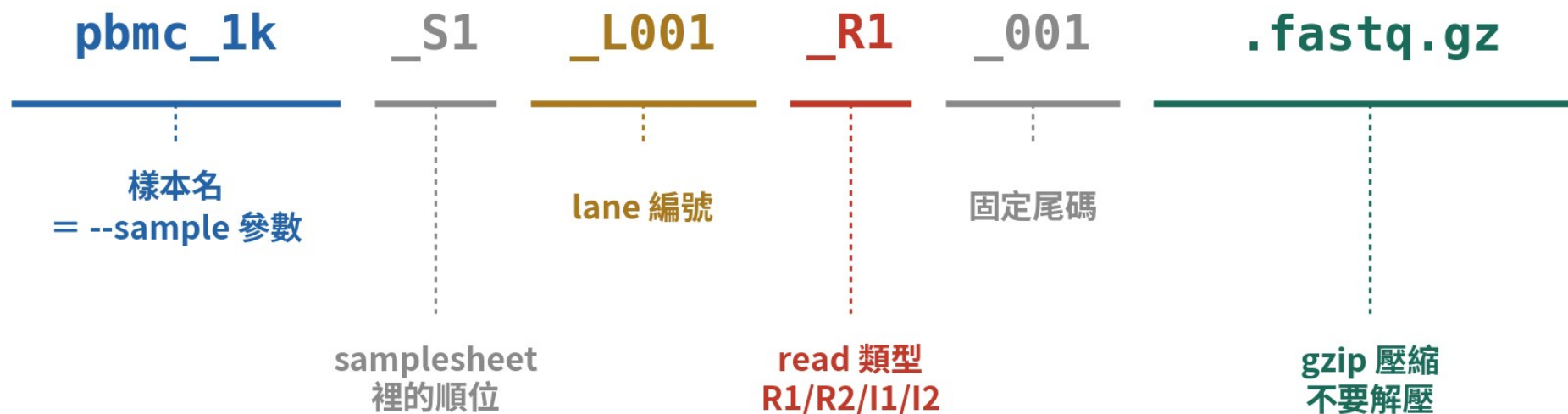
```
# R2 才是 cDNA ; 順便驗證 R1 的讀長是不是 28  
zcatpbm c_1k_v3_S1_L001_R1_001.fastq.gz | head -4000 |  
awk NR % 4 == 2 { print length($0) } | sort | uniq -c
```

1000 28

28 = v3/v4 chemistry、26 = v2。讀長不對，先問定序中心，不要硬跑

檔名也是規格的一部分

cellranger 只認這個格式 (bcl2fastq 命名規則)



少一段、多一段、順序不對——count 都會裝作沒看到這個檔案

cellranger 靠檔名找檔案：樣本名、S 序號、lane、read 類型，一段都不能錯

10x 的 R1/R2 $\neq$ bulk 的 R1/R2

同名不同義，這是從 bulk 轉過來的人最常踩的坑

bulk RNA-seq

R1、R2 都是 cDNA

- paired-end：一條片段的兩端
- 兩個檔案都拿去比對
- 地位對等，掉一個還能單端跑

10x scRNA-seq

R1 是條碼，R2 才是 cDNA

- R1 = cell barcode + UMI，28 bp
- 只有 R2 拿去比對基因組
- 地位不對等，兩個都不能少

到這裡，你已經能讀懂 FASTQ

三個檔案



R1 = 誰
（barcode+UMI）、
R2 = 什麼
（cDNA）、I1/I2
= 哪個樣本

兩個檢查



zcat 看讀長對不對、
檔名合不合規範

一個大坑



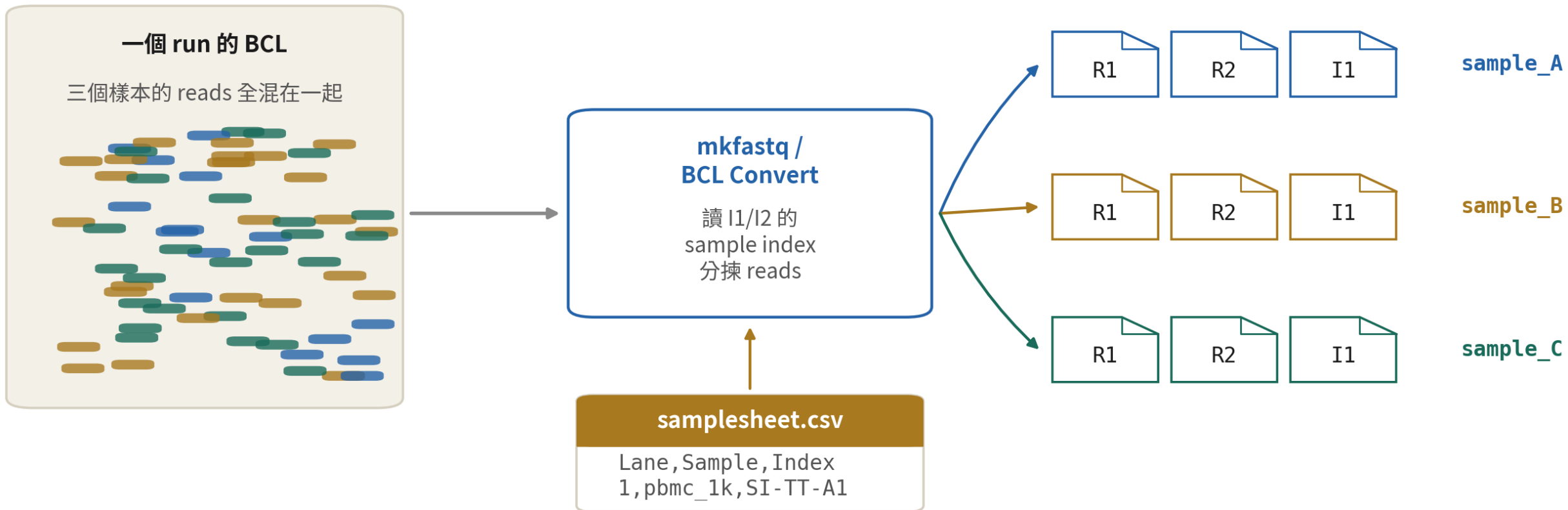
R1/R2 的意義跟
bulk 完全不同，別
憑直覺

04

BCL 到 FASTQ : `mkfastq`

多半輪不到你跑，但你要知道它做了什麼

mkfastq 在做的事：分揀



這一步通常由定序中心代跑——你多半直接拿到右邊這些檔案

讀 sample index，把一個 run 裡混在一起的多個樣本拆成各自的 FASTQ

mkfastq : 指令與 samplesheet

```
# sam plesheet.csv -- 三欄就夠: lane、樣本名、10x index 組名
# Lane,Sam ple,Index
# 1,pbm c_1k,SITT-A1

cellranger mkfastq \
  --run= /data/runs/240801_A00228_0279_BHFW FVDMXX \
  --csv= sam plesheet.csv \
  --output-dir= fastq_out
```

Index 填的是「組名」不是序列；樣本名會直接變成 FASTQ 檔名的開頭

關於 mkfastq ，你真正需要知道的

它是包裝



底層就是 Illumina
的 bcl2fastq ；新版
10x 建議直接用
BCL Convert

多半不用跑



定序中心交件時通常
已經拆好——拿到
FASTQ 就從下一步
開始

但要會驗收



讀長、檔名、檔案大
小三件事，上一節的
檢查照做一遍

05

Reference：預建與自建

九成的人下載就好；剩下一成要會 `mkgtf + mkref`

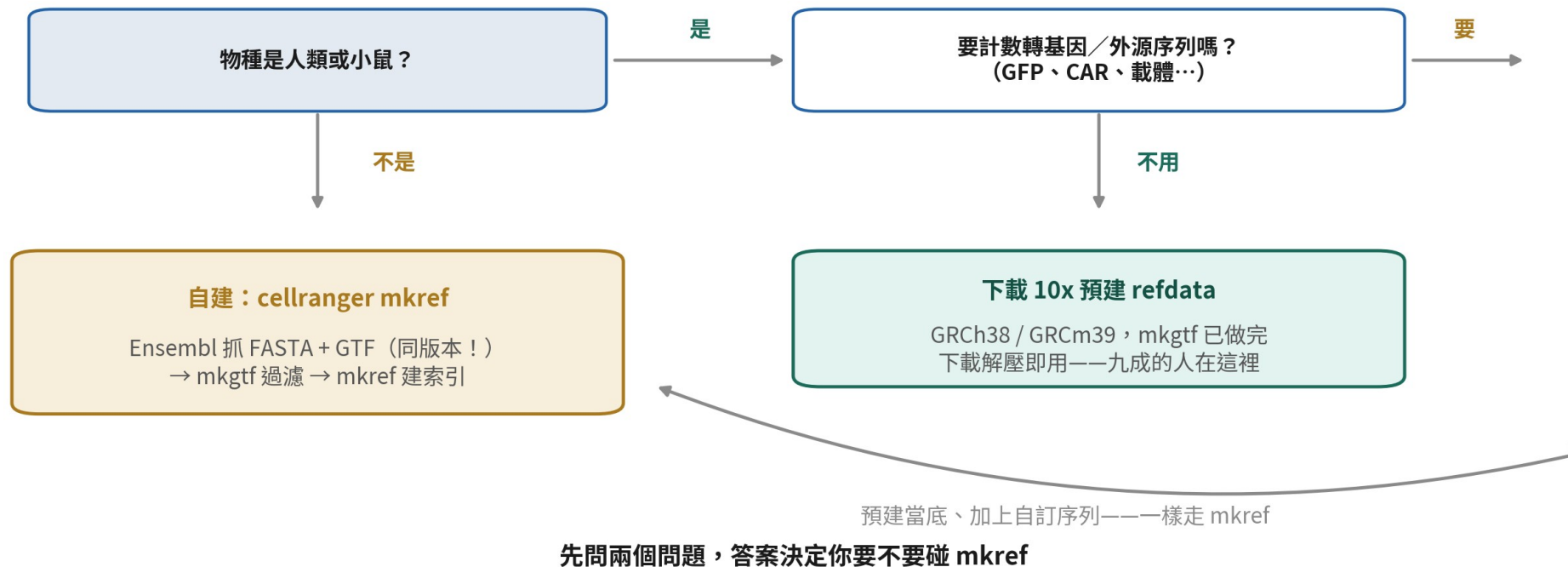
預建 reference：下載即用

```
# 人類 GRCh38 (2024-A, GENCODE v44 注釋), 約 11 GB
curl -O https://cf.10xgenomics.com/supp/cell-exp/
refdata-gex-GRCh38-2024-A.tar.gz
tar -xzf refdata-gex-GRCh38-2024-A.tar.gz
ls refdata-gex-GRCh38-2024-A
```

fasta/	genome.fa	基因組序列
genes/	genes.gtf.gz	注釋 (已過濾)
star/	STAR	比對索引
reference.json		版本與來源紀錄

reference.json 記著基因組與注釋的版本——寫論文方法段時直接抄它

什麼時候必須自建



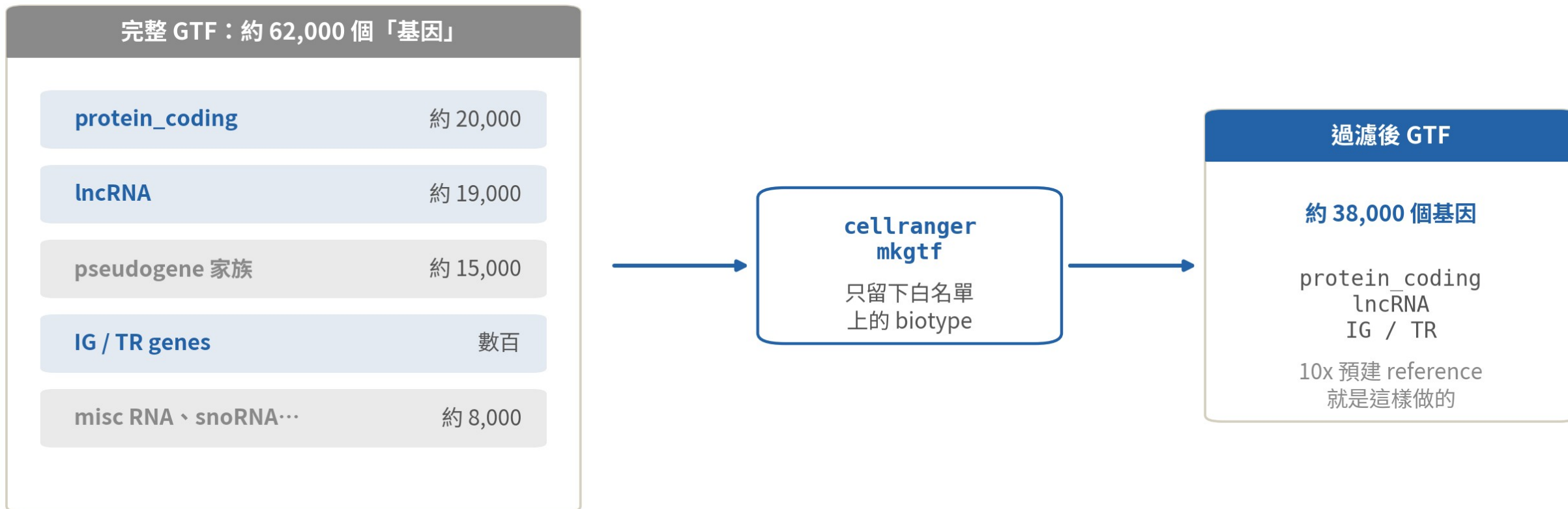
- 人／小鼠、無轉基因 → 預建，順便記下版本
- 非模式物種 → Ensembl 抓同版本 FASTA + GTF 自建
- 要計數 GFP、CAR 等外源序列 → 預建當底加料重建

第一步: mkgtf 過濾注釋

```
cellranger mkgtf \
  Danio_rerio.GRCz11.110.gtf \
  Danio_rerio.GRCz11.110.filtered.gtf \
  --attribute= gene_biotype:protein_coding \
  --attribute= gene_biotype:lncRNA \
  --attribute= gene_biotype:IG_C_gene \
  --attribute= gene_biotype:TR_C_gene
# 以斑馬魚為例；沒列進白名單的 biotype 整類移除
```

--attribute 是白名單：留 protein_coding 與 lncRNA 是 10x 預建的做法

mkgtf 過濾掉的是什麼



被濾掉的類別（偽基因等）從此「不存在」——多序列相似、易吃掉 reads 的雜訊源

偽基因與正主序列高度相似，會把 reads 吸走造成多重比對——濾掉是在保護計數

第二步： mkref 建索引

```
cellranger mkref \  
  --genome=GRCz11_cr \  
  --fasta=Danio_reio.GRCz11.dna.primary_assembly.fa \  
  --genes=Danio_reio.GRCz11.110.filtered.gtf \  
  --nthreads=8 --mem_gb=64  
# 人類基因組約 1-2 小時；輸出資料夾之後餵給 count
```

```
Creating new reference folder at ./GRCz11_cr  
...building STAR index (this can take a while)...  
>>> Reference successfully created! <<<
```

FASTA 與 GTF 必須同一個 release——版本混搭是自建最常見的死法

加轉基因：FASTA 與 GTF 都要動

genome_plus_gfp.fa

```
>chr1 AGCTTACC...  
>chr2 TTGACCAT...  
...
```

```
>GFP ATGGTGAGCAAGGGCGAG...
```

① 序列接到 FASTA 最後面
(多了一條 720 bp 的「迷你染色體」)

genes_plus_gfp.gtf

```
chr1 gene 11869 ...  
chr1 exon 11869 ...  
...
```

```
GFP exon 1 720 gene_id "GFP"
```

② GTF 加上 GFP 的 exon 行
(告訴計數器：這條序列是一個基因)

cellranger
mkref

兩邊都要加，缺一不可——
只加 FASTA，GFP 永遠是 0 (踩雷區見)

GFP 當成一條迷你染色體接上去，GTF 同步加一個基因條目

加 GFP 的實際指令

```
cat refdata/fasta/genome.fa GFP.fa > genome_plus_gfp.fa
```

```
# GTF 加一行 exon (gene_id / transcript_id 都叫 GFP)
echo -e 'GFP\tcustom\texon\t1\t720\t.\t+ \t.\t\
'gene_id "GFP";transcript_id "GFP";gene_name "GFP";'\
>> genes_plus_gfp.gtf
```

```
cellranger mkref --genome=GRCh38_gfp \
--fasta=genome_plus_gfp.fa --genes=genes_plus_gfp.gtf
```

完整可跑的版本在 R/B02_mkref_mkfastq.sh ; 跑完務必驗證 GFP 不是零

cold open 解答

**差兩千多個基因， reads 沒變——
變的是 GTF： 注釋版本與過濾清單。**

所以方法段永遠要寫 reference 版本。別人重跑不出你的結果，第一個要對的就是這行。

06

踩雷區

三個雷，每一個都把後果跑給你看

雷一：把 R1 當 cDNA 送去比對

雷是什麼

憑 bulk 的直覺把 R1/R2 對調餵給流程，或下載時檔案配錯。



為什麼會踩

bulk 的 R1 就是 cDNA，名字一樣，誰想得到意義不同。

踩了會怎樣

cellranger 在 R1 裡找不到合法 barcode，valid barcodes 掉到接近 0，直接中止。

對調之後跑出來的樣子

同一批檔案，只是位置對調

正確：R1=barcode、R2=cDNA

Valid Barcodes

97.2%

Reads Mapped to Genome

91.5%

正常跑完

對調：把 R2 餵給 R1 的位置

Valid Barcodes

0.3%

Reads Mapped to Genome

—

跑到一半直接中止

示意數字

cellranger 在 R1 裡找不到合法 barcode，會在錯誤訊息裡直接點名這件事

好消息：這個雷會大聲報錯。錯誤訊息裡就寫著 barcode 對不上 whitelist

雷二：檔名不合規範，count 拒收

雷是什麼

檔案被改過名（sample.R1.fq.gz 之類），或從雲端下載時被去掉了 _001。



為什麼會踩

cellranger 是用 bcl2fastq 命名規則「猜」檔案的，不是你指哪個檔它讀哪個。

踩了會怎樣

報 No input FASTQs were found，明明檔案就在那裡；改回標準檔名才肯跑。

雷二實錄與解法

```
# [踩雷示範] 檔名少了 _S1_L001 與 _001 :  
# fastq_renam ed/pbm c1k.R1.fq.gz pbm c1k.R2.fq.gz  
cellranger count --id= demo --sam ple= pbm c1k \  
--fastqs= fastq_renam ed --transcriptom e= refdata-...  
  
# 解法: 改回 bc12fastq 命名 (樣本名要與 --sam ple 一致)  
m v pbm c1k.R1.fq.gz pbm c1k_S1_L001_R1_001.fastq.gz  
m v pbm c1k.R2.fq.gz pbm c1k_S1_L001_R2_001.fastq.gz
```

error: No input FASTQs were found for the requested
parameters. Please check your --fastqs and --sam ple.

改名只動檔名、不動內容，安全； --sample 要對上檔名的樣本名那一段

雷三： GFP 加了 FASTA ，忘了 GTF

雷是什麼

自建 reference 時把 GFP 序列接進 FASTA ，但 GTF 沒有加對應條目。



為什麼會踩

mkref 不會抱怨—— FASTA 裡多一條沒有注釋的序列完全合法。

踩了會怎樣

count 正常跑完、無警告， GFP 在每一顆細胞都是 0 。幾週後才發現，實驗重做。

安靜的零，長這樣



count 正常跑完、沒有任何錯誤訊息



每一顆都是 0——整張圖灰的

這種雷最貴：不報錯、只給你一個「安靜的零」，常在幾週後才被發現

一行驗證：`zcat genes.gtf.gz | grep GFP` —— 建完 reference 先跑這行再開 count

踩雷區小結

**大聲的錯誤是恩賜，
安靜的錯誤才要你自己去抓。**

雷一雷二會報錯，照訊息修就好；雷三不會。所以檢查清單要在跑之前過，不是出事之後。

⑤ +

複雜樣本策略室

同一步 reference 決策，換成腫瘤樣本，答案哪裡不同

腫瘤的 reference 決策：病毒也要數得到



做法 = 本集的 GFP 配方：FASTA 加序列、GTF 加條目、mkref 重建——沒加就是雷三的「安靜零」

- 肝癌常與 HBV 相關、鼻咽癌幾乎都帶 EBV——預建 reference 裡沒有病毒序列
- 做法就是本集的 GFP 配方：FASTA 加序列、GTF 加條目、mkref 重建
- 沒加就是雷三：病毒 reads 被安靜丟掉，你永遠看不到感染訊號
- GBM 沒有已知病毒夥伴——同樣是腫瘤，答案不同

PDX 樣本： human + mouse 聯合 reference

示意數字（模擬）

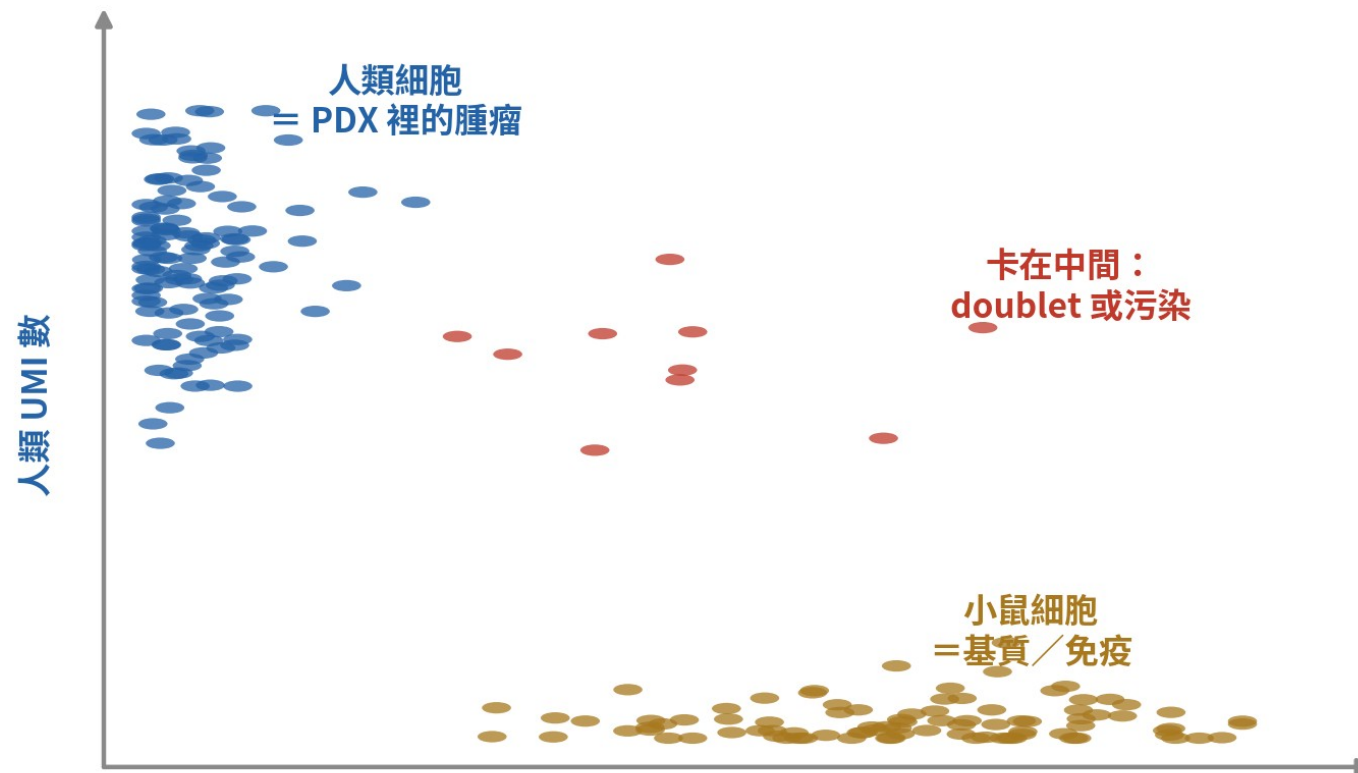
PDX 樣本

病人腫瘤，種在小鼠身上養大

人類：惡性細胞

小鼠：基質＋免疫細胞

→ 用 human + mouse 聯合 reference
(10x 有預建 barnyard 版)



這個 barcode 的小鼠 UMI 數

只用人類 reference = 看不見小鼠細胞，保守序列還會混進人類同源基因的計數

每個 barcode 數人類與小鼠 UMI：貼軸＝單一物種，卡在中間＝doublet 或污染——這張叫 barnyard 圖

reference 決策要往下游想： B16 的 CNV 在等你

基因名來自 GTF



矩陣裡的基因名，就是你今天 count 用的那份 GTF 決定的

CNV 要座標檔



inferCNV 要基因座標檔；版本對不上，對不上的基因整批被丟掉

全專案同一版



腫瘤專案要跨病人比較——從第一個樣本到最後一個都不換版

本集 checklist



說得出 R1 (barcode+UMI) 、R2 (cDNA) 、
I1/I2 (index) 各存什麼



zcat 檢查過讀長：
28 bp 對應 v3/v4 、26 bp 對應 v2



檔名符合
SampleName_S1_L001_R1_001.fastq.gz



人／小鼠且無轉基因 →
用 10x 預建 refdata，記下版本號



自建時 FASTA 與 GTF 同版本，
mkgtf 過濾後才進 mkref



加轉基因時 FASTA 與 GTF 兩邊都加了，
並驗證過 GFP 計數不是零

六格全勾，B3 的 count 才有資格開跑

一句話總結

**FASTQ 回答「誰、什麼、哪個樣本」，
reference 決定「哪些基因數得到」。**

這兩樣備齊、驗過，count 就只是一行指令。

自測 1

10x 3' 資料裡，真正拿去比對基因組的是哪個檔案？

A R1

B R2

C I1

D R1 和 R2 都會比對

答 B。R2 才是 cDNA。R1 只有 28 bp，存的是 cell barcode 加 UMI，是「身分資訊」不是序列內容；I1/I2 是 sample index，分揀完就功成身退。這跟 bulk 的 R1/R2 意義完全不同。

自測 2

你拿到一批小鼠資料，還要計數載體上的 mCherry。reference 怎麼辦？

- A 直接用 10x 預建 mm10/GRCm39
- B Ensembl 重抓 FASTA+GTF 從頭自建
- C 預建當底，FASTA 與 GTF 都加上 mCherry，再 mkref
- D 不用改，mCherry 會自動被偵測

答 C。物種是小鼠，預建可以當底；但 mCherry 不在裡面，必須 FASTA 加序列、GTF 加條目、重跑 mkref。只加一邊都不行——只加 FASTA 就是雷三，計數永遠是零。

自測 3

同一批 FASTQ 換了 reference，測到的基因數差兩千多。最可能的原因是？

- A 定序品質不穩定
- B GTF 注釋版本與過濾清單不同
- C cellranger 版本的隨機性
- D 細胞數不同

答 B。reads 沒變，能變的只有 reference。FASTA 序列多年不動，動的是 GTF：新版注釋收錄更多基因（尤其 lncRNA），filter 白名單不同也會整類增減。所以方法段一定要寫 reference 版本。

下集預告

B3 : cellranger count 實戰與 web_summary 判讀

FASTQ 齊了、reference 齊了——count 一鍵下去，跑出十幾個檔案。
哪個該先開？哪些數字代表「這批資料能用」？下一集，一格一格看懂 web_summary。